



MANUAL TEST GUIDE · MODULE 11 OF 11

# Tracking & Visibility Functional-Flow Test Walkthroughs

Scenario-driven QA guide for **logistics.tracking** — the read-mostly visibility surface that turns the equipment movement-event stream into customer-facing milestones, an internal timeline, an ETA history, exception alerts, and email. It owns **no** source-of-truth events; it reacts to five upstream ones.

**How to use:** Work the scenarios top to bottom against a tenant loaded with `--with-demo=all`. Each step gives a precondition, an action, the expected result, and a ✓ **checkpoint** to tick. Green = happy path; amber/red steps deliberately trip a guard to confirm it holds.

# 01 What Tracking does & how to read this guide

Tracking answers **"where is my cargo?"** in customer language. It is a **projection**: it listens to events that upstream modules already emit and derives `tracking.milestone`, `tracking.timeline.entry`, `tracking.leg.event`, an ETA chain, and exception alerts. It never writes a movement event and never writes shipment status directly — `source_event_id` is read-only, and status advances are **emitted** for the shipments module to apply.

8

COMMANDS TO EXERCISE

3

TEST SCENARIOS

5

UPSTREAM EVENTS IT REACTS TO

0

SOURCE EVENTS IT OWNS

## The five upstream events Tracking listens to

Nothing here is triggered by a Tracking button — the module is **event-driven**. These listeners create/activate the activation and rebuild projections.

```
ede.record.created (logistics.shipment) → initialise activation (initialized) + auto-subscribe customer
shipment.ready_for_execution → activate the shipment activation (active) + rebuild
shipment.leg_added_removed → project milestones from movement events
consolidation.console.confirmed → console-level activation
equipment.movement.event.recorded → on-event milestone projection (the truth source)
```

## The state machines you will observe

```
activation.stage : initialized → active → suspended → closed
milestone.status : planned → completed | delayed | missed (also: actual)
exception.status : open → acknowledged → resolved → closed
eta.snapshot : append-only chain · one is_current=true · original_eta immutable
```

FIVE SUBJECT LEVELS

- **shipment** — the demo direct FCL export
- **house / master** — consolidation grain
- **console** — grouped console
- **leg** — per-leg multimodal projection

THE THREE MASTERS THAT DRIVE PROJECTION

- `milestone.type.master` — vocabulary + `source_event_codes` map
- `exception.code.master` — alert / delay / ETA reasons
- `status.mapping.master` — milestone → shipment-status advance

## The scenarios in this guide

| Scenario                    | What it proves                                                                                                                             | Key commands                                                                 | Section   |
|-----------------------------|--------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------|-----------|
| A — Activation & projection | Event-driven activation lifecycle; milestone chain + timeline projected from movement events; idempotent activate; status-advance emission | <code>activation.activate</code> / <code>suspend</code> / <code>close</code> | §02       |
| B — ETA, exceptions & email | Append-only ETA chain + threshold-driven delay alert; exception open → ack → resolve → close; email via foundation.notifications           | <code>eta.snapshot.publish</code> , <code>exception.alert.*</code>           | §03       |
| C — Manual + legs + guards  | Role-gated manual update; per-leg multimodal projection + roll-up; subscription; the negative guards                                       | <code>milestone.manual_update</code>                                         | §04 · §05 |

Demo fixtures you start from (loaded by `--with-demo=all` )

Over shipment `shipments.demo_shipment_globex_001` (INMUM → SGSIN): an **active** activation `tracking.demo_activation_globex` ; a milestone chain **Booked** · **Picked Up** · **Departed** (completed) → **Arrived** (planned); two timeline entries; an ocean leg event ( `MAERSK SEMARANG / 526W` ); a published ETA snapshot (06-27 06:00 → 18:00, +12h); an **open** `MILESTONE_DELAY` exception; and an email subscription for **Globex Inc.**

## 02 Activation lifecycle & milestone projection

**Goal:** prove Tracking activates itself from upstream events and projects a customer-facing milestone chain from the movement-event stream — without any Tracking-side data entry. **Role:** Ops monitor. **Start:** the Globex shipment activation is already `active` in demo.

| # | Action                                                                                                                 | Expected result                                                                                                                                                                                                                                                                                                          | ✓ Checkpoint                                                        | Trace     |
|---|------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------|-----------|
| 1 | Open the <b>Tracking</b> app; find the activation for the Globex shipment.                                             | One <code>tracking.activation</code> at <code>activation_level=shipment</code> , <code>stage=active</code> , <code>activation_trigger=ready_for_execution</code> . It was created by the <code>ede.record.created</code> listener and activated by <code>shipment.ready_for_execution</code> — not by a Tracking action. | Activation exists & is active; no Tracking record was hand-created. | TRK-01    |
| 2 | Re-run <b>Activate</b> ( <code>tracking.activation.activate</code> ) on the same activation.                           | Idempotent: returns <code>noop:true</code> with <code>stage</code> unchanged. Re-firing on an already-active (or closed) subject is a no-op — no error, no state churn.                                                                                                                                                  | Second activate is a no-op, not a duplicate.                        | TRK-BR-15 |
| 3 | Read the milestone chain for the shipment.                                                                             | Four <code>tracking.milestone</code> rows ordered by <code>display_sequence</code> : <b>Booked · Picked Up · Departed</b> <b>Origin = completed</b> ; <b>Arrived at Destination = planned</b> . Each completed row carries <code>actual_datetime</code> .                                                                | Chain projected; sequence + statuses match demo.                    | TRK-02    |
| 4 | Inspect a completed milestone's <code>source_event_id</code> and <code>source</code> .                                 | Milestones are derived from movement events; <code>source_event_id</code> is <b>read-only</b> and Tracking never mutates the event. The event → milestone map lives on the type's <code>source_event_codes</code> CSV (e.g. <code>DEPARTED</code> → <code>DEPARTED_ORIGIN</code> ).                                      | Provenance points back to the event; nothing owns the event here.   | TRK-02    |
| 5 | Open the internal <b>timeline</b> for the shipment.                                                                    | Chronological <code>tracking.timeline.entry</code> rows mirror the milestones (Booked; "Departed Origin (INMUM)" with the 4.5h-late remark), each linking its <code>milestone_id</code> .                                                                                                                                | Timeline mirrors the chain in event order.                          | TRK-14    |
| 6 | Record a new <b>equipment.movement.event</b> for the shipment's equipment usage (e.g. an ARRIVED gate event upstream). | The <code>equipment.movement.event.recorded</code> listener runs <code>project_from_event</code> and <b>upserts</b> the matching milestone (idempotent per type) — only because the activation is <code>active</code> . Projection is skipped for inactive subjects.                                                     | New event advances the projection automatically.                    | TRK-02    |
| 7 | Confirm a status advance was requested (not written by Tracking).                                                      | Completing <b>Departed Origin</b> emits <code>tracking.status.advance_requested</code> with <code>target_status=in_transit</code> (mapping requires current <code>booked</code> ). The <b>shipments</b> module owns the write — producer-owns-the-contract, no bridge.                                                   | Advance is emitted, not applied by Tracking.                        | TRK-13    |

### ✓ Scenario A pass criteria

The activation reached **active** purely from upstream events; the milestone chain + timeline are present with read-only event provenance; a fresh movement event advanced the projection; and status change was **emitted** for shipments to apply.

### ⚠ Suspend vs close

`tracking.activation.suspend` pauses projection (stage → `suspended`) reversibly; `close` is terminal and is auto-fired when a **terminal** milestone (DELIVERED) completes (TRK-BR-16). Projections only run while `stage=active`.

## 03 ETA chain, exception alerts & the email path

**Goal:** exercise the append-only ETA history, the threshold-driven delay exception, its open → resolve lifecycle, and the customer email — whose **delivery** is owned by `foundation.notifications`, not Tracking. **Start:** the demo carries a published current ETA snapshot and an open `MILESTONE_DELAY` alert.

| # | Action                                                                                                                               | Expected result                                                                                                                                                                                                                                                                                                | ✓ Checkpoint                                            | Trace           |
|---|--------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------|-----------------|
| 1 | Open the <b>ETA snapshot</b> chain for the shipment.                                                                                 | The current snapshot shows <code>original_eta=06-27 06:00</code> (immutable), <code>current_eta=06-27 18:00</code> , <code>eta_delta_hours</code> computed, <code>is_current=true</code> .                                                                                                                     | <b>Original ETA immutable; exactly one current row.</b> | TRK-12          |
| 2 | Append a <b>new</b> ETA revision (create another snapshot for the same subject).                                                     | The pre-create hook copies <code>original_eta</code> forward and sets <code>previous_eta</code> ; the post-create hook flips the <b>prior</b> snapshot's <code>is_current</code> to false. History is append-only.                                                                                             | <b>Prior snapshot demoted; new one is current.</b>      | TRK-BR-31/32    |
| 3 | Make the revision's delta ≥ <code>tracking.eta.delay_alert_threshold_hours</code> (default 6).                                       | A <code>MILESTONE_DELAY</code> exception is opened — or the existing open one is <b>reused</b> , not duplicated (idempotent per subject+code).                                                                                                                                                                 | <b>Threshold breach raises/reuses one alert.</b>        | TRK-BR-29       |
| 4 | Run <b>Publish</b> ( <code>tracking.eta.snapshot.publish</code> ) on the current snapshot.                                           | <code>is_published=true</code> and <code>published_at</code> stamped. Only published snapshots reach customers (portal is Phase 2).                                                                                                                                                                            | <b>Snapshot marked published.</b>                       | TRK-12          |
| 5 | Open the exception <b>Departure delayed at INMUM</b> ; run <b>Acknowledge</b> ( <code>tracking.exception.alert.acknowledge</code> ). | <code>status</code> open → <code>acknowledged</code> .                                                                                                                                                                                                                                                         | <b>Alert acknowledged.</b>                              | TRK-11          |
| 6 | Run <b>Resolve</b> ( <code>tracking.exception.alert.resolve</code> ) with a <code>resolution_remarks</code> .                        | <code>status=resolved</code> , <code>resolved_at</code> stamped, remarks saved; an <code>exception_closed</code> audit entry is written.                                                                                                                                                                       | <b>Resolved + audited.</b>                              | TRK-11 · TRK-15 |
| 7 | Run <b>Close</b> ( <code>tracking.exception.alert.close</code> ).                                                                    | <code>status=closed</code> ; a second <code>exception_closed</code> audit entry is written.                                                                                                                                                                                                                    | <b>Alert reaches terminal closed.</b>                   | TRK-11          |
| 8 | Confirm the customer <b>email</b> for the exception.                                                                                 | Opening a customer-visible alert fires the <code>tracking.exception.created</code> key → recipient resolution over <code>tracking.subscription</code> → a <code>notification.send</code> dispatch to <b>foundation.notifications</b> (email channel). A <code>notification_sent</code> audit entry records it. | <b>Email dispatched via notifications; audited.</b>     | TRK-10          |

### ✓ Scenario B pass criteria

ETA history stays append-only with an immutable original and a single current row; a threshold breach opens (or reuses) exactly one delay alert; the alert walks open → acknowledged → resolved → closed with audit at each close; and the email is dispatched through `foundation.notifications`, not Tracking itself.

### ⚠ Delivery ownership

Tracking owns the **subscription**, the **event keys**, recipient resolution, and template content — **not** delivery. If email fails, look at `foundation.notifications`. Phase 1 honours the `email` channel only; other channels are stored but skipped at send time.

## 04 Manual updates, per-leg tracking & subscriptions

**Goal:** drive an audited manual milestone update, then prove multimodal journeys project **per leg** — each leg independent, rolled up to the shipment — and confirm subscription-driven recipients. **Start:** the demo ocean leg event and the Globex email subscription are present.

| # | Action                                                                                                                                                                                                          | Expected result                                                                                                                                                                                                                             | ✓ Checkpoint                                               | Trace           |
|---|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------|-----------------|
| 1 | Run <b>Manual Update</b> ( <code>tracking.milestone.manual_update</code> ) on the <b>Arrived</b> milestone → <code>new_status=completed</code> with an <code>actual_datetime</code> .                           | An append-only <code>tracking.manual_update</code> row is created, then applied to the milestone (status → completed). The change fires the milestone side-effects and writes an audit entry.                                               | <b>Manual row created; milestone updated + audited.</b>    | TRK-05          |
| 2 | Observe the downstream effects of completing <b>Arrived</b> .                                                                                                                                                   | Because the type has <code>notify_on_reach=true</code> , a <code>tracking.milestone.reached</code> notification fires; status automation emits an advance to <code>arrived</code> (mapping requires <code>in_transit</code> ).              | <b>Reach notified; advance emitted.</b>                    | TRK-10 · TRK-13 |
| 3 | Read the manual row's <code>update_role</code> .                                                                                                                                                                | The acting role is denormalised onto the audit ledger ( <code>tracking_updater</code> / <code>tracking_admin</code> ); hard role gating is enforced by the RBAC permission rows on the command.                                             | <b>Acting role recorded for audit.</b>                     | TRK-BR-21       |
| 4 | Open the <b>leg event</b> for the ocean main carriage.                                                                                                                                                          | A <code>tracking.leg.event</code> at <code>leg_type=ocean_main_carriage</code> , <code>leg_sequence=1</code> , <code>conveyance_reference="MAERSK SEMARANG / 526W"</code> , projected read-only from its movement event.                    | <b>Per-leg event present with conveyance ref.</b>          | TRK-03          |
| 5 | For a door-to-door journey, add leg events across modes — <b>road_pickup</b> → <b>ocean_main_carriage</b> → <b>transshipment</b> → <b>road_delivery</b> — each with its own <code>leg_sequence</code> and mode. | Each leg is projected <b>independently</b> : a delay on one leg does not change another leg's status. Legs roll up to the shipment milestone when <code>tracking.leg.rollup_to_shipment</code> is on.                                       | <b>Legs sequenced &amp; independent; roll-up honoured.</b> | TRK-03          |
| 6 | Run a <code>manual_update</code> targeting a leg via <code>leg_event_id</code> (instead of <code>milestone_id</code> ).                                                                                         | The leg event's <code>status</code> is updated to the new status with <code>source=manual</code> ; the update is still audited.                                                                                                             | <b>Per-leg manual update applied.</b>                      | TRK-05          |
| 7 | Inspect the <b>subscription</b> for the Globex customer.                                                                                                                                                        | One active <code>tracking.subscription</code> at <code>subscription_level=shipment</code> , <code>channels=email</code> . It gates recipients per event via <code>notify_on_*</code> flags. Only <code>email</code> is honoured in Phase 1. | <b>Customer subscribed; email channel active.</b>          | TRK-10          |

### ⚠ The anti-pattern to catch

A multimodal journey is **one** shipment with many `tracking.leg.event` rows keyed by `leg_id` + `leg_sequence` — never a flat single record holding two modes, and never per-mode duplicate tracking models. A one-leg delay must not silently mutate a sibling leg.

## 05 Negative checks & append-only immutability

Half of testing a visibility surface is confirming it **refuses** bad or lossy input. Each row below trips a real `ValueError`; the message shown is the exact string from the code.

| # | Action                                                                                                                           | Expected refusal (exact message)                                                                                                                              | ✓       | Trace        |
|---|----------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------|---------|--------------|
| 1 | <code>manual_update</code> with a <code>new_status</code> but <b>no</b> <code>milestone_id</code> or <code>leg_event_id</code> . | "manual_update requires milestone_id or leg_event_id"                                                                                                         | Blocked | TRK-05       |
| 2 | <code>manual_update</code> → <code>completed</code> without an <code>actual_datetime</code> .                                    | "a completed status requires actual_datetime (TRK-05)"                                                                                                        | Blocked | TRK-05       |
| 3 | <code>manual_update</code> → <code>delayed</code> without a <code>delay_reason_id</code> .                                       | "a delayed status requires delay_reason_id (TRK-05)"                                                                                                          | Blocked | TRK-05       |
| 4 | <code>manual_update</code> → <code>delayed</code> / <code>missed</code> without <code>remarks</code> .                           | "a delayed/missed status requires remarks (TRK-05)"                                                                                                           | Blocked | TRK-05       |
| 5 | Edit any business field on an existing <code>tracking.manual.update</code> row.                                                  | "manual updates are append-only — record a new update instead (TRK-BR-22)"                                                                                    | Blocked | TRK-BR-22    |
| 6 | Edit — or delete — a <code>tracking.audit.entry</code> row.                                                                      | "tracking audit entries are append-only and immutable (TRK-BR-35)" / "...cannot be deleted (TRK-BR-35)"                                                       | Blocked | TRK-BR-35    |
| 7 | Delete a <b>standard-locked</b> milestone type (e.g. <code>BOOKED</code> ).                                                      | "standard milestone types cannot be deleted — deactivate instead (TRK-BR-04)"                                                                                 | Blocked | TRK-BR-04    |
| 8 | Delete a standard-locked <b>exception code</b> / <b>status mapping</b> .                                                         | "standard exception codes cannot be deleted — deactivate instead (TRK-BR-40)" / "standard status mappings cannot be deleted — deactivate instead (TRK-BR-27)" | Blocked | TRK-BR-40/27 |
| 9 | Dispatch <code>activation.activate</code> / <code>exception.alert.acknowledge</code> with <b>no</b> target id.                   | "activation command requires a target activation_id" / "exception command requires a target exception_id"                                                     | Blocked | guard        |

### ⚠ Not a `ValueError` — an idempotent no-op

Re-running `tracking.activation.activate` (or `close`) on an already **active** / **closed** activation does **not** raise. It returns `{ ..., "noop": true }` and leaves state untouched (TRK-BR-15). Don't expect an error here — expect a clean no-op.

### Config toggles worth flipping during the run

| Config key                                                     | Default | Effect to verify                                                                   |
|----------------------------------------------------------------|---------|------------------------------------------------------------------------------------|
| <code>tracking.eta.delay_alert_threshold_hours</code>          | 6       | ETA delta ≥ this opens/reuses a <code>MILESTONE_DELAY</code> alert.                |
| <code>tracking.eta.auto_publish</code>                         | false   | On → new snapshots are published on create (skip step B4).                         |
| <code>tracking.manual_update.require_attachment_for_pod</code> | false   | On → completing a POD milestone without an attachment is refused.                  |
| <code>tracking.status_automation.enabled</code>                | true    | Gates emission of <code>tracking.status.advance_requested</code> (§02 step 7).     |
| <code>tracking.activation.auto_on_ready</code>                 | true    | Off → <code>shipment.ready_for_execution</code> initialises but does not activate. |

### ✓ Full-module pass criteria

All three scenarios green: event-driven activation + projection (A), append-only ETA + exception lifecycle + email (B), audited manual updates with independent per-leg projection + subscriptions (C) — and every guard in this section confirmed to refuse, with the activate no-op understood as intentional.

*Traceability: TRK-\* IDs map to the Tracking & Visibility BRD; TRK-BR-\* are the business-rule guards asserted directly in `logistics.tracking` source. Cross-module effects (email delivery, `shipment-status` writes) are verified in the **foundation.notifications** and **Shipments** guides respectively.*